基于改进YOLOv11的多类型道路异常事件检测系统

硕士毕业论文实现方案

一、研究背景与意义

1.1 研究背景

高速公路和城市道路上的异常事件（如抛洒物遗撒、机动车违停、车辆逆行）是导致二次交通事故的重要原因。据统计，约 **30%** 的高速公路事故与路面异物有关，违停和逆行更是直接威胁其他车辆安全。传统的人工巡检效率低、覆盖面有限，无法满足全天候实时监控需求。因此，基于深度学习的道路异常事件自动检测系统具有重大现实意义。

1.2 研究意义

维度	意义
安全性	实时检测三类异常事件，提前预警，减少二次事故
经济性	自动化检测替代人工巡检，降低运维成本
实时性	基于改进YOLOv11实现端到端检测，满足实时监控需求
视频理解	从单帧检测升级为视频级事件判定，符合实际监控场景
学术价值	提出三个创新模块并通过消融实验验证有效性

1.3 核心目标

> **一个骨干网络 + 时序视频推理**：输入一段道路监控视频，通过逐帧检测与视频级时序聚合，判定该视频中包含哪种异常事件：
>
> 1. 🚧 **抛洒物**（路面散落物体）
> 2. 🚚 **机动车违停**（违规停放车辆）
> 3. 🔄 **逆行**（车辆逆向行驶）

> **关键认知**：每个原始视频本身就代表一个事件，系统通过分析同一视频中多帧检测结果的时序统计特征，做出视频级别的事件类型判定——而不仅仅是单帧检测。

1.4 当前数据集情况

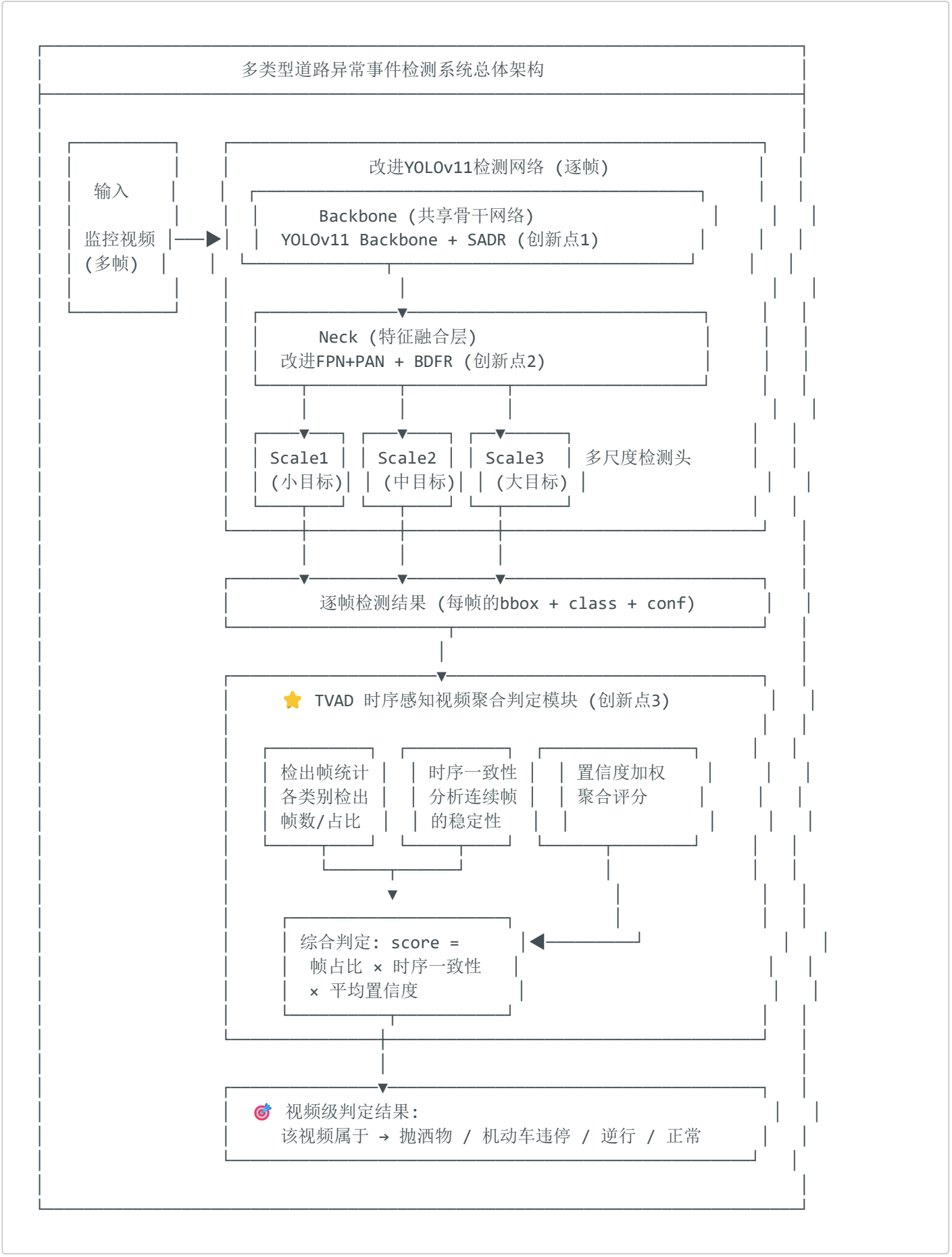
类别	数据形式	数量	存放路径
抛洒物	MP4视频	~180个	`data/抛洒物/`
机动车违停	MP4视频	~100个	`data/机动车违停/`
逆行	MP4视频	~50个	`data/逆行/`
拥堵（备用）	MP4视频	~125个	`data/拥堵/`
人员入侵（备用）	MP4视频	~50个	`data/人员入侵/`
抛洒物图片	JPG图片	~800张	`data/images/`

1.5 技术挑战与解决思路

挑战	描述	解决思路
多任务统一	三类异常事件差异大，需统一检测框架	共享骨干 + 多任务检测头
小目标检测	抛洒物在监控视角下占比极小	创新点1：尺度自适应动态路由模块(SADR)
复杂背景干扰	道路结构化背景造成高误检	创新点2：背景解耦特征精炼模块(BDFR)
视频级事件判定	单帧检测无法可靠判定事件类型，需时序聚合	创新点3：时序感知视频聚合判定模块(TVAD)
运动状态判别	逆行需要理解车辆运动方向	时序帧差特征 + 视频级运动方向一致性分析
数据泄露风险	同一视频的帧可能分到不同集合，导致评估虚高	按视频分组的数据集划分策略
实时性要求	监控场景需 ≥25FPS	YOLOv11轻量化 + 重参数化推理

二、系统总体架构

2.1 整体框架



2.2 技术路线总览

```
数据准备 ──▶ 模型设计 ──▶ 训练优化 ──▶ 实验验证 ──▶ 系统部署
  │            │            │            │            │
  ├ 视频抽帧    ├ YOLOv11基线  ├ 多任务联合   ├ 消融实验    ├ PyQt5界面
  │ (视频源追溯) ├ 创新点1:SADR ├ 数据增强     │ (A0~A4)    ├ 视频级推理
  ├ 数据标注    ├ 创新点2:BDFR ├ 超参搜索    ├ 对比实验    ├ 实时检测
  ├ 数据增强    ├ 创新点3:TVAD ├ 学习率调度   ├ 可视化分析   ├ 模型导出
  └ 按视频划分  └ 视频级推理    └ 视频级评估   └ 论文撰写    └ 时间线报告
```

2.3 核心设计理念："帧检测 → 视频判定" 两阶段架构

本系统的核心认知：**每段视频对应一个交通事件**，最终目标是判定"这段视频属于哪种异常"。

```
┌──────────────────────────────────────────────────────────────┐
│  第一阶段：空间感知（逐帧）                                    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  改进YOLOv11 → 每一帧独立检测目标                       │    │
│  │  • SADR：自适应尺度路由，解决目标大小差异                  │    │
│  │  • BDFR：背景解耦精炼，抑制道路背景干扰                   │    │
│  │  输出：每帧 → [(class_id, conf, bbox), ...]            │    │
│  └──────────────────────────────────────────────────────┘    │
│                              │                                 │
│                              ▼                                 │
│  第二阶段：时序聚合（视频级）                                   │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  TVAD模块 → 聚合同一视频所有帧的检测结果                  │    │
│  │  • 检出帧比例统计：某类别检出帧数 / 总帧数                 │    │
│  │  • 时序一致性分析：连续帧检出是否稳定                     │    │
│  │  • 置信度加权评分：综合 score 排序                       │    │
│  │  输出：视频 → 事件类型（抛洒物/违停/逆行/正常）           │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

为什么需要两阶段？

问题	仅单帧检测	帧检测 + 视频聚合
偶发误检	单帧误检直接输出错误结果	误检帧占比低，被聚合策略过滤掉
逆行判定	单帧无法判断运动方向	通过连续帧的目标位置变化推断方向
违停判定	难区分临时停靠与违停	多帧持续检出 → 确认违停
结果可靠性	依赖单帧模型精度	时序投票机制天然抗噪

三、数据准备方案

3.1 数据处理流程

```
原始数据 (视频 → 每个视频代表一个事件)
        │
        ▼
    ┌──────────────┐
    │  Step 1:     │
    │  视频抽帧    │  按类别差异化采样 (抛洒物6fps/违停4fps/逆行8fps)
    │  (保留视频源) │  帧命名: category_v01_0001.jpg (保留视频来源)
    └──────────────┘
        │
        ▼
    ┌──────────────┐
    │  Step 2:     │
    │  质量筛选    │  帧差去重(阈值15) + 模糊过滤(阈值80), 严格保质
    └──────────────┘
        │
        ▼
    ┌──────────────┐
    │  Step 3:     │
    │  数据标注    │  使用LabelImg标注YOLO格式
    └──────────────┘
        │
        ▼
    ┌──────────────┐
    │  Step 4:     │
    │  数据增强    │  翻转/旋转/Mosaic/MixUp
    └──────────────┘
        │
        ▼
    ┌──────────────┐
    │  Step 5:     │
    │  数据集划分  │  按视频分组划分 (同一视频帧不跨split)
    └──────────────┘  训练集:验证集:测试集 = 7:2:1
```

> ⚠ **关键约束**：数据集划分以**视频**为最小单位，同一视频的所有帧必须归入同一个 split（train/val/test），防止数据泄露。

3.2 视频抽帧脚本设计

```python
# 文件: scripts/extract_frames.py
"""
视频抽帧脚本
- 对三类视频数据按类别差异化采样率抽帧
- 帧命名保留视频来源: category_v{视频序号}_{帧内序号}.jpg
- 严格帧差去重 (阈值15) + 模糊过滤 (阈值80)
- 高采样率 + 严格去重 = 质量优先
"""
import cv2
import os
import numpy as np
from pathlib import Path

# 按类别差异化采样率
DEFAULT_CATEGORY_FPS = {
    'debris': 6,            # 抛洒物: 小目标、形态多变, 密采覆盖更多尺度
```

```
    'illegal_parking': 4,   # 违停：需前后帧对比判定，提高时序密度
    'retrograde': 8,        # 逆行：运动速度快，高密度防漏检
}

def extract_frames(video_dir, output_dir, category, fps_sample=6,
                   similarity_threshold=15.0, blur_threshold=80.0):
    """
    从视频目录中提取关键帧
    帧命名格式: category_v{视频序号:02d}_{帧内序号:04d}.jpg
    → 保留视频来源信息，用于后续按视频分组划分数据集
    """
    os.makedirs(output_dir, exist_ok=True)
    video_files = sorted(Path(video_dir).glob("*.mp4"))

    total_count = 0
    for vid_idx, vid_path in enumerate(video_files, start=1):
        cap = cv2.VideoCapture(str(vid_path))
        fps = cap.get(cv2.CAP_PROP_FPS) or 25.0
        interval = max(1, int(fps / fps_sample))

        prev_gray = None
        frame_count = 0
        idx = 0
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            if idx % interval == 0:
                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

                # 严格帧差去重
                if prev_gray is not None:
                    diff = cv2.absdiff(gray, prev_gray)
                    if np.mean(diff) < similarity_threshold:
                        idx += 1
                        continue

                # 模糊过滤
                blur = cv2.Laplacian(gray, cv2.CV_64F).var()
                if blur < blur_threshold:
                    idx += 1
                    continue

                # 保存帧 (命名保留视频来源)
                fname = f"{category}_v{vid_idx:02d}_{frame_count:04d}.jpg"
                cv2.imwrite(os.path.join(output_dir, fname), frame)
                prev_gray = gray
                frame_count += 1

            idx += 1
        cap.release()
        total_count += frame_count

    print(f"[{category}] 共提取 {total_count} 帧 (来自 {len(video_files)} 个视频)")
    return total_count
```

3.3 数据标注规范

标注格式：YOLO格式（txt文件，每行一个目标）

```
<class_id> <x_center> <y_center> <width> <height>
```

类别定义（3个类别）：

class_id	类别名	英文名	说明
0	抛洒物	debris	路面散落物体（纸箱、塑料袋、石块、车辆零件等）
1	机动车违停	illegal_parking	在禁停区域/应急车道违规停放的车辆
2	逆行	retrograde	在车道内逆向行驶的车辆

3.4 数据集配置文件

```yaml
# 文件: dataset/road_anomaly.yaml
path: ./dataset
train: images/train
val: images/val
test: images/test

nc: 3  # 类别数量
names:
  0: debris            # 抛洒物
  1: illegal_parking   # 机动车违停
  2: retrograde        # 逆行
```

3.5 数据增强策略

增强方法	参数	用途
Mosaic	4图拼接	增加小目标上下文信息
MixUp	alpha=0.5	提升模型鲁棒性
随机翻转	p=0.5 (水平)	增加样本多样性
HSV扰动	H:0.015, S:0.7, V:0.4	适应光照变化
随机缩放	scale=0.5~1.5	适应不同距离的目标
随机透视	perspective=0.001	模拟不同摄像头角度

3.6 预估数据量

类别	原始视频数	预估抽帧数	有效标注帧
抛洒物	~180	~5400帧	~3000帧
机动车违停	~100	~3000帧	~2000帧
逆行	~50	~1500帧	~1000帧
images图片	-	~800张	~800张
合计	**~330**	**~10700**	**~6800**

四、模型设计方案

4.1 基础模型：YOLOv11

选用 **YOLOv11（Ultralytics YOLO11）** 作为基础框架：

特性	说明
框架	Ultralytics最新YOLO系列（2024年发布）
改进	相比YOLOv8提升了C2f为C3k2模块、引入C2PSA注意力
性能	更高的mAP，更快的推理速度
生态	完善的训练/推理/导出工具链

YOLOv11基础结构：

```
Input(640×640×3)
        │
        ▼
    ┌──────────────────────────┐
    │        Backbone          │
    │  Conv → C3k2 → Conv → C3k2 │
    │  → Conv → C3k2 → Conv    │
    │  → C3k2 → SPPF → C2PSA   │
    └──────────────────────────┘
            │
            │  P3   P4   P5
            ▼
    ┌──────────────────────────┐
    │          Neck            │
    │    FPN (自顶向下融合)      │
    │    PAN (自底向上融合)      │
    └──────────────────────────┘
            │
```

```
            │
            ▼
    ┌──────────────────────────┐
    │   Detection Head         │
    │   多尺度解耦检测头          │
    │   (P3: 小目标)            │
    │   (P4: 中目标)            │
    │   (P5: 大目标)            │
    └──────────────────────────┘
```

4.2 ⭐ 创新点1：尺度自适应动态路由模块 (SADR)

4.2.1 问题分析——为什么现有方法不够

道路监控场景中的三类目标尺度差异极大：

- **抛洒物**：远端摄像头下仅 8×8 ~ 32×32 像素，是典型小目标
- **违停车辆**：64×64 ~ 256×256 像素，中等目标
- **逆行车辆**：近端可能超过 256×256 像素，大目标

现有方法的核心缺陷：

方法	做法	问题
Inception/ASPP	多分支固定膨胀率并行	所有空间位置使用相同的感受野混合比例，无法适应每个位置的目标尺度
SKNet	通道级别的核选择	全图共享同一组通道权重，整张图只有一个"最优尺度"选择
FPN/PAN	多尺度特征金字塔	不同层之间存在语义鸿沟，简单的上/下采样+拼接丢失了尺度对齐信息
可变形卷积DCN	学习采样偏移	只是改变采样位置，没有显式建模"该位置适合什么尺度"

核心洞察：一张道路监控图像中，不同空间位置的目标尺度是不同的（近处目标大、远处目标小），理想的特征提取应该在**每个像素位置**独立地选择最适合的感受野尺度——这正是现有方法缺失的能力。

4.2.2 模块设计

SADR (Scale-Adaptive Dynamic Routing) 的核心创新：

1. 引入一个轻量级的**尺度预测子网络**，为输入特征图的每个空间位置生成一个**逐像素的尺度亲和力图（Scale Affinity Map）**

2. 用该亲和力图作为软路由权重，在多个不同感受野的特征分支之间进行**逐像素的加权融合**
3. 尺度预测网络本身不使用任何固定池化，而是通过**条带池化（Strip Pooling）**保留方向性空间信息——这对道路场景非常重要（车道具有明确的方向性）

```
                 输入特征 F_in (B, C, H, W)
                         |
        ┌────────────────┼──────────────────────────┐
        |                |                          |
   ┌────▼────┐   ┌────▼────┐   ┌────▼────┐    ┌──────────▼──────────┐
   | Branch-S|   |Branch-M |   |Branch-L |    | Scale Predictor     |
   | k=3,d=1|   | k=3,d=3 |   | k=3,d=5 |    |  (尺度预测网络)      |
   | 小感受野|   | 中感受野 |   | 大感受野|    |                     |
   | (局部)  |   | (区域)  |   | (全局)  |    | StripPool_H         |
   └────────┘   └────────┘   └────────┘    | StripPool_W         |
        |            |            |         | → 1×1Conv           |
        |      F_s   |     F_m    |   F_l   | → Softmax(dim=1)    |
        |            |            |         | → W (B,3,H,W)       |
        |            |            |         └─────────────────────┘
        |            |            |                   |
        |            |            |             W_s, W_m, W_l
        |            |            |           (逐像素尺度权重)
        |            |            |                   |
        └────────────┴────────────┴───────────────────┘
                                 |
                   ┌─────────────▼──────────────┐
                   | 逐像素加权融合:             |
                   | F_out(b,:,h,w) =           |
                   |   W_s(b,h,w)·F_s +         |
                   |   W_m(b,h,w)·F_m +         |
                   |   W_l(b,h,w)·F_l           |
                   └────────────────────────────┘
                                 |
                   ┌─────────────▼──────────────┐
                   |   1×1 Conv + BN            |
                   |   + 残差连接               |
                   └────────────────────────────┘
                                 |
                            输出特征
```

4.2.3 关键创新点剖析

创新1：逐像素尺度路由 (Per-Pixel Scale Routing)

与 SKNet 等方法在通道级别（全图一个权重）选择核不同，SADR 为特征图的**每个空间位置 (h, w)** 独立预测一组尺度权重 $[W_s, W_m, W_l]$。这意味着：

- 图像远端区域（小目标）自动获得更大的 W_s 权重（偏向小感受野，保留精细特征）
- 图像近端区域（大目标）自动获得更大的 W_l 权重（偏向大感受野，捕获全局语义）
- 权重经过 Softmax 归一化，保证 $W_s + W_m + W_l = 1$

创新2：条带池化驱动的尺度预测 (Strip-Pooling-Driven Prediction)

尺度预测网络的设计是关键。我们不使用全局平均池化（会丢失空间信息）或常规卷积（感受野不够大），而是采用**水平条带池化 + 垂直条带池化**的组合：

$$F_h = \text{AvgPool}_{1\times W}(F_{in}) \quad \text{(水平条带: 捕获垂直位置信息)}$$

$$F_v = \text{AvgPool}_{H\times 1}(F_{in}) \quad \text{(垂直条带: 捕获水平位置信息)}$$

这一设计利用了道路场景的先验知识：道路在图像中通常呈纵向延伸，目标尺度沿纵向（近大远小）有明显的空间规律。条带池化能以极低的计算代价（仅 $O(C \cdot H + C \cdot W)$）捕获这种全局空间分布信息。

创新3：膨胀卷积替代大核 (Dilated Conv vs Large Kernel)

三个分支不使用 3×3/5×5/7×7 不同大小的卷积核（Inception 做法），而是统一使用 **3×3 卷积配不同膨胀率**（d=1/3/5），这样：

- 参数量完全相同（均为 3×3=9 个参数/核）
- 感受野分别为 3×3、7×7、11×11，覆盖更大范围
- 避免大核卷积的高计算开销

4.2.4 与现有方法的本质区别

对比维度	SKNet	ASPP	DCNv2	SADR (本文)
路由粒度	通道级 (全图共享)	无路由 (固定并行)	采样点级	**像素级** (每点独立)
路由机制	Softmax通道权重	无	学习偏移量	**尺度亲和力图**
空间感知	无 (全局池化)	无	有 (偏移)	**有 (条带池化)**
任务先验	通用	通用	通用	**道路场景感知**
计算开销	中	高	高	**低**

4.2.5 代码实现

```python
# 文件: models/modules/sadr.py
"""
SADR: Scale-Adaptive Dynamic Routing Module (创新点1)
尺度自适应动态路由模块

核心创新:
1. 逐像素尺度路由 — 每个空间位置独立选择最优感受野组合
2. 条带池化驱动预测 — 利用道路场景方向性先验
3. 统一膨胀卷积 — 等参数量覆盖更大感受野范围
"""
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```python
class StripPooling(nn.Module):
    """
    条带池化模块
    分别沿水平和垂直方向进行全局平均池化，
    保留方向性空间结构信息（对道路场景至关重要）
    """
    def __init__(self, in_channels, mid_channels=None):
        super().__init__()
        mid_channels = mid_channels or max(in_channels // 4, 32)

        # 水平条带: (B,C,H,W) → (B,C,H,1) → 1×1Conv
        self.h_pool = nn.AdaptiveAvgPool2d((None, 1))  # 沿W方向池化
        self.h_conv = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, 1, bias=False),
            nn.BatchNorm2d(mid_channels),
            nn.SiLU(inplace=True)
        )

        # 垂直条带: (B,C,H,W) → (B,C,1,W) → 1×1Conv
        self.v_pool = nn.AdaptiveAvgPool2d((1, None))  # 沿H方向池化
        self.v_conv = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, 1, bias=False),
            nn.BatchNorm2d(mid_channels),
            nn.SiLU(inplace=True)
        )

        # 融合: 广播相加后映射
        self.fuse = nn.Sequential(
            nn.Conv2d(mid_channels, mid_channels, 1, bias=False),
            nn.BatchNorm2d(mid_channels),
            nn.SiLU(inplace=True)
        )

    def forward(self, x):
        _, _, H, W = x.shape
        # 水平条带: (B,mid,H,1) → 广播到 (B,mid,H,W)
        h_feat = self.h_conv(self.h_pool(x))           # (B, mid, H, 1)
        # 垂直条带: (B,mid,1,W) → 广播到 (B,mid,H,W)
        v_feat = self.v_conv(self.v_pool(x))           # (B, mid, 1, W)
        # 广播相加 → (B, mid, H, W)
        combined = h_feat + v_feat                      # 自动广播
        return self.fuse(combined)                      # (B, mid, H, W)


class ScalePredictor(nn.Module):
    """
    尺度预测子网络
    为每个空间位置预测3个尺度分支的亲和力权重
    输出: (B, num_branches, H, W)，经Softmax归一化
    """
    def __init__(self, in_channels, num_branches=3):
        super().__init__()
        self.strip_pool = StripPooling(in_channels)
        mid = max(in_channels // 4, 32)

        # 从条带池化特征 → 逐像素尺度权重
        self.predictor = nn.Sequential(
            nn.Conv2d(mid, num_branches, 1, bias=True)
            # bias=True 允许默认偏向某个尺度
        )
```

```python
        self.num_branches = num_branches

    def forward(self, x):
        strip_feat = self.strip_pool(x)              # (B, mid, H, W)
        logits = self.predictor(strip_feat)            # (B, 3, H, W)
        weights = F.softmax(logits, dim=1)             # 归一化为概率
        return weights  # (B, 3, H, W)


class DilatedBranch(nn.Module):
    """膨胀卷积分支: 3×3 Conv with dilation, 等参数量不同感受野"""
    def __init__(self, channels, dilation):
        super().__init__()
        self.conv = nn.Conv2d(
            channels, channels, 3,
            padding=dilation, dilation=dilation,
            groups=channels, bias=False   # 深度卷积, 参数量 = 9*C
        )
        self.bn = nn.BatchNorm2d(channels)
        self.pw = nn.Conv2d(channels, channels, 1, bias=False)
        self.pw_bn = nn.BatchNorm2d(channels)
        self.act = nn.SiLU(inplace=True)

    def forward(self, x):
        x = self.act(self.bn(self.conv(x)))
        x = self.act(self.pw_bn(self.pw(x)))
        return x


class SADR(nn.Module):
    """
    Scale-Adaptive Dynamic Routing Module
    尺度自适应动态路由模块

    输入: (B, C, H, W) 特征图
    输出: (B, C, H, W) 特征图 (尺度自适应增强)

    工作原理:
    1. 三个膨胀卷积分支 (d=1,3,5) 提取不同感受野特征
    2. 尺度预测网络为每个像素位置生成路由权重
    3. 逐像素加权融合三个分支的输出
    4. 残差连接
    """
    def __init__(self, channels, num_branches=3):
        super().__init__()

        # 三个不同感受野的分支
        # d=1: 感受野 3×3  (小目标 — 抛洒物)
        # d=3: 感受野 7×7  (中目标 — 逆行车辆)
        # d=5: 感受野 11×11 (大目标 — 违停车辆)
        self.branches = nn.ModuleList([
            DilatedBranch(channels, dilation=1),
            DilatedBranch(channels, dilation=3),
            DilatedBranch(channels, dilation=5),
        ])

        # 尺度预测子网络
        self.scale_predictor = ScalePredictor(channels, num_branches)

        # 输出投影
        self.proj = nn.Sequential(
```

```
            nn.Conv2d(channels, channels, 1, bias=False),
            nn.BatchNorm2d(channels)
        )

    def forward(self, x):
        # 1. 计算各分支特征
        branch_feats = [branch(x) for branch in self.branches]
        # 每个: (B, C, H, W)

        # 2. 预测逐像素尺度路由权重
        weights = self.scale_predictor(x)  # (B, 3, H, W)

        # 3. 逐像素加权融合
        # weights[:, i:i+1, :, :] → (B, 1, H, W)，广播乘以 (B, C, H, W)
        fused = sum(
            weights[:, i:i+1, :, :] * branch_feats[i]
            for i in range(len(self.branches))
        )

        # 4. 投影 + 残差连接
        return self.proj(fused) + x
```

4.2.6 嵌入位置

将 SADR 嵌入 YOLOv11 的 **Backbone 末端**（SPPF之后、C2PSA之前），让骨干网络输出的特征自适应调整感受野：

```
# Backbone中SPPF之后加入SADR
- [-1, 1, SPPF, [1024, 5]]        # 9  原始SPPF
- [-1, 1, SADR, [1024]]            # 10 ← 创新点1: 尺度自适应路由
- [-1, 2, C2PSA, [1024]]           # 11 原始C2PSA
```

4.2.7 正向作用分析

效果	分析
小目标提升	远端像素位置自动路由到小感受野分支，避免大核模糊小目标
大目标不退化	近端像素位置自动路由到大感受野分支，获取充分语义
场景自适应	不同道路场景（直道/弯道/匝道）自动调整尺度分配
轻量高效	深度膨胀卷积 + 条带池化，新增参数量 < 3%

4.3 ⭐ 创新点2：背景解耦特征精炼模块 (BDFR)

4.3.1 问题分析——为什么标准注意力机制不够

道路监控具有一个独特的场景特性：**背景高度结构化且可预测**。路面、车道线、护栏、中央分隔带等元素在不同帧、不同时段中反复出现，形成了一种"道路背景模式"。而异常事件（抛洒物、违停车辆、逆行车辆）恰恰是**偏离这种正常模式的异常前景**。

现有注意力机制的局限：

方法	做法	关键缺陷
SE-Net	全局池化→通道权重	不区分前景/背景，只做通道重要性排序
CBAM	通道+空间注意力串联	空间注意力只看统计量(max/avg)，不理解"什么是背景"
ECA	局部通道交互	完全没有空间建模能力
GAM	通道-空间并行	对前景和背景无差别对待

核心洞察：如果模型能显式地学习到"正常道路背景长什么样"，然后将特征中"与背景相似的部分"抑制、"与背景偏离的部分"增强，就能大幅提升异常目标的检测能力。这在本质上不同于CBAM等通用注意力——它引入了**场景语义先验**。

4.3.2 模块设计

BDFR (Background-Decoupled Feature Refinement) 的核心思想：

1. 维护一组**可学习的背景原型向量（Background Prototypes）**，它们在训练过程中自动学习到"正常道路背景特征"的表示
2. 对每个空间位置的特征，计算其与背景原型的**偏离度（Deviation Score）**
3. 偏离度高的位置（更可能是异常目标）→ 特征增强；偏离度低的位置（更可能是背景）→ 特征抑制
4. 使用EMA（指数移动平均）策略在训练中稳定更新背景原型

```
          输入特征 F_in (B, C, H, W)
                 │
                 ├──────────────────────────────────┐
                 │                                  │
        ┌────────▼────────┐              ┌──────────▼──────────┐
        │ 特征投影         │              │ 背景原型 P (K个, 可学习)  │
        │ F_proj = ϕ(F_in) │              │ P ∈ R^{K × C'}       │
        │ (B,C',H,W)       │              │ (训练中通过EMA持续更新)   │
        └──────────────────┘              └──────────────────────┘
                 │                                  │
                 │                                  │
                 │          ┌──────────────────────┘
                 │          │
        ┌────────▼──────────▼──────────┐
        │                              │
        │  计算偏离度 (Deviation):      │
        │                              │
        │  对每个位置(h,w):             │
        │    d(h,w) = min_k            │
        │      ||F_proj(h,w) - P_k|| │
```

```
│                                │
│  即：该位置特征与最近背景       │
│  原型的距离                    │
│                                │
│  D = normalize(d) → [0,1]      │
│  (偏离度图)                    │
└────────────────────────────────┘
                │
                │
                │  D (B, 1, H, W)
                │
┌───────────────▼────────────────┐
│  异常感知注意力:               │
│                                │
│  A = σ(Conv(D))                │
│  (将偏离度映射为注意力权重)    │
│                                │
│  F_ref = A ⊙ F_in              │
│  (高偏离度 → 增强)             │
│  (低偏离度 → 抑制)             │
└────────────────────────────────┘
                │
                │
┌───────────────▼────────────────┐
│  残差融合:                     │
│  F_out = F_ref + F_in          │
└────────────────────────────────┘
                │
                │
            输出特征
```

4.3.3 关键创新点剖析

创新1：可学习背景原型 (Learnable Background Prototypes)

模块内部维护 K 个（默认 K=8）可学习的原型向量 $\mathbf{P} = [\mathbf{p}_1, \mathbf{p}_2, ..., \mathbf{p}_K] \in \mathrm{R}^{K \times C'}$，它们通过反向传播自动学习道路背景的多种模式（路面纹理、车道线、护栏等不同背景成分）。

$$\mathbf{p}_k \leftarrow \mathbf{p}_k - \eta \frac{\partial \mathrm{L}}{\partial \mathbf{p}_k}$$

与聚类中心不同，这些原型不需要K-Means初始化，而是通过端到端训练自然收敛到背景特征分布的关键模式上。

创新2：偏离度驱动的注意力 (Deviation-Driven Attention)

传统注意力（SE/CBAM）通过统计量（均值/最大值）来衡量特征重要性，这是一种"绝对重要性"度量。BDFR采用"**相对偏离度**"——即特征与背景原型的距离——来衡量一个位置是否包含异常目标：

$$d(h, w) = \min_{k=1,...,K} // \mathbf{f}(h, w) - \mathbf{p}_k //_2$$

- d 大 → 该位置特征远离所有已知背景模式 → 很可能是异常目标 → **增强**
- d 小 → 该位置特征接近某个背景模式 → 很可能是路面/车道线 → **抑制**

这种设计使得注意力具有明确的**语义含义**（偏离背景的程度），而不仅仅是数值上的重要性。

创新3：EMA原型更新策略

训练过程中采用指数移动平均(EMA)使原型更新更稳定：

$$\mathbf{p}_k^{(t)} = \beta \cdot \mathbf{p}_k^{(t-1)} + (1-\beta) \cdot \bar{\mathbf{f}}_k^{(t)}$$

其中 $\beta = 0.99$，$\bar{\mathbf{f}}_k^{(t)}$ 是当前batch中被分配到第 k 个原型的特征均值。EMA防止原型被单个batch的噪声所干扰。

4.3.4 与现有方法的本质区别

对比维度	SE-Net	CBAM	异常检测方法	**BDFR (本文)**
注意力依据	通道统计量	通道+空间统计量	重构误差	**背景偏离度**
背景建模	无	无	有但独立模型	**端到端集成**
语义可解释性	弱	弱	强	**强**
计算方式	全局池化→FC	池化→Conv	编码器-解码器	**原型距离**
参数开销	小	小	大(独立模型)	**小(仅K个原型)**
目标场景	通用	通用	异常检测专用	**检测内注意力**

4.3.5 代码实现

```python
# 文件: models/modules/bdfr.py
"""
BDFR: Background-Decoupled Feature Refinement Module (创新点2)
背景解耦特征精炼模块

核心创新:
1. 可学习背景原型向量 — 端到端学习道路背景模式
2. 偏离度驱动注意力 — 基于"与背景的距离"而非统计量
3. EMA原型稳定更新 — 防止训练震荡

本质区别于CBAM/SE/GAM: 引入了"背景是什么"的显式建模,
而不仅仅是"哪里重要"的隐式估计
"""
import torch
import torch.nn as nn
import torch.nn.functional as F


class BDFR(nn.Module):
    """
    Background-Decoupled Feature Refinement Module
    背景解耦特征精炼模块

    Args:
```

```
        channels: 输入特征通道数
        num_prototypes: 背景原型数量 K (默认8, 表示8种背景模式)
        proj_dim: 投影维度 (默认channels//4, 降维减少距离计算开销)
        ema_momentum: EMA动量系数 (默认0.99)
    """
    def __init__(self, channels, num_prototypes=8, proj_dim=None,
                 ema_momentum=0.99):
        super().__init__()
        self.channels = channels
        self.num_prototypes = num_prototypes
        self.proj_dim = proj_dim or max(channels // 4, 32)
        self.ema_momentum = ema_momentum

        # 特征投影: C → proj_dim (降维, 减少距离计算量)
        self.feature_proj = nn.Sequential(
            nn.Conv2d(channels, self.proj_dim, 1, bias=False),
            nn.BatchNorm2d(self.proj_dim),
            nn.SiLU(inplace=True)
        )

        # 可学习背景原型: K个 proj_dim 维向量
        # 初始化为标准正态, 训练中会自动收敛到背景模式
        self.prototypes = nn.Parameter(
            torch.randn(num_prototypes, self.proj_dim) * 0.1
        )

        # EMA原型 (不参与梯度, 仅用于稳定更新)
        self.register_buffer(
            'prototype_ema',
            torch.randn(num_prototypes, self.proj_dim) * 0.1
        )

        # 偏离度 → 注意力权重的映射
        # 使用小型卷积网络而非简单sigmoid, 增强表达能力
        self.deviation_to_attn = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1, bias=False),
            nn.BatchNorm2d(16),
            nn.SiLU(inplace=True),
            nn.Conv2d(16, 1, 1, bias=True),
            nn.Sigmoid()
        )

    @torch.no_grad()
    def _update_prototypes_ema(self, features):
        """
        EMA更新背景原型（仅在训练时调用）
        将当前batch的特征根据最近原型分组, 更新对应原型
        """
        if not self.training:
            return

        B, C, H, W = features.shape
        # 展平空间维度: (B, C, H, W) → (B*H*W, C)
        feat_flat = features.permute(0, 2, 3, 1).reshape(-1, C)

        # 计算与每个原型的距离: (N, K)
        dist = torch.cdist(feat_flat.unsqueeze(0),
                           self.prototypes.unsqueeze(0)).squeeze(0)

        # 找到每个特征最近的原型
        assignments = dist.argmin(dim=1)  # (N,)
```

```python
        # 对每个原型，计算分配给它的特征的均值
        for k in range(self.num_prototypes):
            mask = (assignments == k)
            if mask.sum() > 0:
                cluster_mean = feat_flat[mask].mean(dim=0)
                self.prototype_ema[k] = (
                    self.ema_momentum * self.prototype_ema[k] +
                    (1 - self.ema_momentum) * cluster_mean
                )

        # 将EMA值同步回可学习参数(软更新)
        self.prototypes.data = (
            0.9 * self.prototypes.data +
            0.1 * self.prototype_ema
        )

    def _compute_deviation(self, proj_features):
        """
        计算每个空间位置到最近背景原型的偏离度

        Args:
            proj_features: (B, proj_dim, H, W) 投影后的特征

        Returns:
            deviation_map: (B, 1, H, W) 归一化偏离度图
        """
        B, C, H, W = proj_features.shape

        # 展平: (B, C, H, W) → (B, H*W, C)
        feat = proj_features.flatten(2).permute(0, 2, 1)  # (B, N, C)

        # 原型: (K, C) → (1, K, C)
        protos = self.prototypes.unsqueeze(0)  # (1, K, C)

        # 计算每个位置到每个原型的L2距离: (B, N, K)
        # 使用矩阵运算避免循环
        dist = torch.cdist(feat, protos)  # (B, N, K)

        # 取到最近原型的距离作为偏离度: (B, N)
        min_dist, _ = dist.min(dim=2)  # (B, N)

        # 归一化到 [0, 1] (使用batch内的min-max)
        d_min = min_dist.min(dim=1, keepdim=True)[0]  # (B, 1)
        d_max = min_dist.max(dim=1, keepdim=True)[0]  # (B, 1)
        deviation = (min_dist - d_min) / (d_max - d_min + 1e-6)

        # 恢复空间形状: (B, N) → (B, 1, H, W)
        deviation_map = deviation.view(B, 1, H, W)

        return deviation_map

    def forward(self, x):
        """
        Args:
            x: (B, C, H, W) 输入特征
        Returns:
            (B, C, H, W) 精炼后的特征
        """
        residual = x
```

```
        # 1. 特征投影 (降维)
        proj = self.feature_proj(x)  # (B, proj_dim, H, W)

        # 2. EMA更新原型 (仅训练时)
        self._update_prototypes_ema(proj.detach())

        # 3. 计算偏离度图
        deviation_map = self._compute_deviation(proj)  # (B, 1, H, W)

        # 4. 偏离度 → 注意力权重
        attn = self.deviation_to_attn(deviation_map)  # (B, 1, H, W)
        # attn值高 → 远离背景 → 可能是目标 → 增强
        # attn值低 → 接近背景 → 可能是路面 → 抑制

        # 5. 注意力加权 + 残差连接
        # 使用 (1 + attn) 而非 attn，保证不会完全丢失背景信息
        refined = x * (1.0 + attn)

        return refined + residual
```

4.3.6 嵌入位置

将 BDFR 嵌入 YOLOv11 的 **Neck 层各特征融合节点之后**，精炼融合后的特征：

```
# Neck中每个C3k2之后加入BDFR
head:
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 6], 1, Concat, [1]]
  - [-1, 2, C3k2, [512, False]]
  - [-1, 1, BDFR, [512]]            # ← 创新点2: 背景解耦精炼

  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 4], 1, Concat, [1]]
  - [-1, 2, C3k2, [256, False]]
  - [-1, 1, BDFR, [256]]            # ← 创新点2: 背景解耦精炼

  # ... PAN路径同理
```

4.3.7 正向作用分析

效果	分析
降低误检率	路面纹理/车道线/护栏等背景特征被抑制，减少背景误报
提升召回率	异常目标特征被增强，减少漏检
场景泛化	背景原型学习多种道路模式，不依赖特定道路场景
可解释性强	偏离度图可直接可视化，解释"模型认为哪里有异常"
参数极少	仅 $K \times C'$ 个原型参数（如8×64=512个浮点数）

4.4 改进后的完整网络结构

```yaml
# 文件: models/yolov11-road-anomaly.yaml
# 基于YOLOv11改进的道路异常事件检测网络
# 创新点1: SADR (骨干网络末端)
# 创新点2: BDFR (Neck各特征融合节点后)
# 创新点3: TVAD (推理阶段视频级聚合判定, 非模型结构)

nc: 3  # 类别数: debris, illegal_parking, retrograde
scales:
  m: [0.67, 0.75, 768]  # 使用medium规模

# Backbone
backbone:
  - [-1, 1, Conv, [64, 3, 2]]           # 0  P1/2
  - [-1, 1, Conv, [128, 3, 2]]          # 1  P2/4
  - [-1, 2, C3k2, [256, False, 0.25]]   # 2
  - [-1, 1, Conv, [256, 3, 2]]          # 3  P3/8
  - [-1, 2, C3k2, [512, False, 0.25]]   # 4
  - [-1, 1, Conv, [512, 3, 2]]          # 5  P4/16
  - [-1, 2, C3k2, [512, True]]          # 6
  - [-1, 1, Conv, [1024, 3, 2]]         # 7  P5/32
  - [-1, 2, C3k2, [1024, True]]         # 8
  - [-1, 1, SPPF, [1024, 5]]            # 9
  - [-1, 1, SADR, [1024]]               # 10  ★ 创新点1: 尺度自适应路由
  - [-1, 2, C2PSA, [1024]]              # 11

# Neck + Head
head:
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]   # 12
  - [[-1, 6], 1, Concat, [1]]                     # 13
  - [-1, 2, C3k2, [512, False]]                   # 14
  - [-1, 1, BDFR, [512]]                          # 15  ★ 创新点2: 背景解耦精炼

  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]    # 16
  - [[-1, 4], 1, Concat, [1]]                     # 17
  - [-1, 2, C3k2, [256, False]]                   # 18
  - [-1, 1, BDFR, [256]]                          # 19  ★ 创新点2: 背景解耦精炼

  - [-1, 1, Conv, [256, 3, 2]]                    # 20
  - [[-1, 15], 1, Concat, [1]]                    # 21
  - [-1, 2, C3k2, [512, False]]                   # 22
  - [-1, 1, BDFR, [512]]                          # 23  ★ 创新点2: 背景解耦精炼

  - [-1, 1, Conv, [512, 3, 2]]                    # 24
  - [[-1, 11], 1, Concat, [1]]                    # 25
  - [-1, 2, C3k2, [1024, True]]                   # 26
  - [-1, 1, BDFR, [1024]]                         # 27  ★ 创新点2: 背景解耦精炼

  - [[19, 23, 27], 1, Detect, [nc]]               # 28  三尺度检测
```

4.5 多任务检测头设计

虽然三类异常本质上都是目标检测任务，但在逻辑上对应不同的检测语义。实现方式为**单检测头多类别输出**——由统一的 `Detect` Head 同时预测 3 个类别，通过最终的 class confidence 区分输出是抛洒物、违停还是逆行：

```
Detect Head 输出:

  ┌──────────────────────────────────────┐
  │  每个检测框预测:                      │
  │    - bbox: (x, y, w, h)              │
  │    - objectness: 目标置信度           │
  │    - class_probs: [P_debris,         │
  │                    P_illegal_parking, │
  │                    P_retrograde]     │
  │                                      │
  │  最终输出 = argmax(class_probs)       │
  │    → 该检测框属于哪类异常             │
  └──────────────────────────────────────┘
```

4.6 ⭐ 创新点3：时序感知视频聚合判定模块 (TVAD)

4.6.1 问题分析——为什么单帧检测不够

本系统的数据形态是**视频**而非图片集：每段视频记录了一个完整的交通事件（一段抛洒物堆积的过程、一辆违停车辆的持续状态、一辆车逆行的全过程）。然而传统目标检测方法（包括 YOLOv11）只做单帧检测，存在以下核心缺陷：

问题	仅单帧检测	后果
偶发误检	某帧因光照/遮挡产生的误检无法校正	误报率高
逆行判定	单帧无法区分正常行驶与逆行	逆行漏检率高
违停确认	无法区分临时停靠(几秒)与违停(持续)	误判临时停车为违停
结果粒度	只能输出"这一帧检出了什么"	无法回答"这段视频是什么事件"

核心洞察：检测系统的最终输出应该是**视频级别的事件判定**——"这段视频记录的是一起抛洒物事件"——而不是一堆离散的单帧检测框。这需要对同一视频的多帧检测结果进行**时序聚合**。

4.6.2 模块设计

TVAD (Temporal-aware Video Aggregation Decision) 在推理阶段工作，接收同一视频所有帧的检测结果，通过三个维度的时序分析做出视频级判定：

```
                输入: 同一视频的 T 帧检测结果
                  [{frame_1: detections}, ..., {frame_T: detections}]
                                  |
                  ┌───────────────┼───────────────┐
                  |               |               |
  ┌───────────────▼───────────┐ ┌─▼─────────────┐ ┌─▼─────────────┐
  | 维度1:                    | | 维度2:        | | 维度3:        |
  | 检出帧比例统计            | | 时序一致性    | | 置信度加权    |
  |                           | | 分析          | |               |
  | 对每个类别c:              | | 连续N帧窗口   | | 该类别所有    |
  | r_c = 检出帧数            | | 内检出次数    | | 检出的平均    |
  |     / 总帧数              | | 的平滑度      | | 置信度 conf_c |
  |                           | | τ_c ∈[0,1]    | |               |
  └───────────────┬───────────┘ └───────┬───────┘ └───────┬───────┘
                  |                     |                 |
                  └─────────────────────┘                 |
                                  |                       |
                                  ▼                       |
                                                          |
      ┌──────────────────────────────────┐                |
      |  综合评分:                       |                |
      |  S_c = r_c × τ_c                 |◀───────────────┘
      |        × conf_c                  |
      |                                  |
      |  其中:                           |
      |   r_c: 检出帧比例 (持续性)       |
      |   τ_c: 时序一致性 (稳定性)       |
      |   conf_c: 平均置信度             |
      └──────────────────────────────────┘
                        |
                        |
      ┌──────────────────────────────────┐
      |  判定规则:                       |
      |                                  |
      |  1. 过滤: r_c ≥ 10%              |
      |     且检出帧 ≥ 3帧               |
      |  2. 排序: argmax(S_c)            |
      |  3. 输出: 最高分类别             |
      |     = 该视频事件类型             |
      └──────────────────────────────────┘
                        |
                        ▼
            🎯 视频判定: "该视频为XX事件"
```

4.6.3 关键创新点剖析

创新1：三维度融合评分 (Triple-Dimension Scoring)

不同于简单的多数投票(majority voting)或平均置信度，TVAD综合三个正交维度进行评分：

$$S_c = r_c \cdot \tau_c \cdot \bar{c}_c$$

- **检出帧比例** r_c（持续性）：该类别检出帧数 / 视频总帧数。消除偶发误检（误检的 r_c 通常 < 5%）
- **时序一致性** τ_c（稳定性）：在滑动窗口 W 内，连续检出的帧占比的平均值。区分"零星检出"和"持续检出"

- **平均置信度** $\bar{c}_c$：所有检出的置信度均值。抑制低置信度的弱检出

三个维度缺一不可——仅用 r_c 无法区分"高频零星误检"和"持续稳定检出"，仅用 $\bar{c}_c$ 无法处理"少量帧高置信"的偶发情况。

创新2：时序一致性分析 (Temporal Consistency Analysis)

$$\tau_c = \frac{1}{T-w+1} \sum_{t=1}^{T-w+1} 1\left[\sum_{i=t}^{t+w-1} d_c(i) \geq \lfloor w/2 \rfloor\right]$$

其中 $d_c(i) \in \{0,1\}$ 表示第 i 帧是否检出类别 c，w 为滑动窗口大小。τ_c 衡量的是"在多少比例的时间窗口中，该类别被稳定检出"。

这一设计对三类事件的意义：

- **抛洒物**：静态物体→连续帧持续检出→τ 高
- **违停车辆**：持续存在→连续帧持续检出→τ 高
- **逆行车辆**：运动中→在某段时间窗口内连续检出→τ 中等
- **误检**：零星出现→大部分窗口内检出不连续→τ 低（被过滤）

创新3：自适应判定阈值

不同类别使用不同的判定阈值，适配其事件特性：

类别	最低检出帧数	最低帧比例	逻辑
抛洒物	≥ 3帧	≥ 10%	静态目标，少量帧即可确认
违停	≥ 5帧	≥ 15%	需确认持续停放，非临时停靠
逆行	≥ 3帧	≥ 8%	运动事件，出现即判定（但防误检）

4.6.4 与现有方法的本质区别

对比维度	多数投票	平均置信度	时序模型(LSTM)	**TVAD (本文)**
时序建模	无	无	有(隐状态)	**有(显式统计)**
可解释性	强	弱	弱(黑盒)	**强(各维度可视化)**
额外训练	不需要	不需要	需要训练	**不需要**
鲁棒性	低(受误检影响)	中	中(需大量数据)	**高(三维度互补)**
部署开销	零	零	高(序列模型)	**极低(统计计算)**

TVAD的核心优势：**无需额外训练、零推理开销、可解释性强、天然适配视频级任务**。

4.6.5 代码实现

```python
# 文件: scripts/inference.py (video_inference函数)
"""
TVAD: Temporal-aware Video Aggregation Decision
时序感知视频聚合判定模块

工作在推理阶段，接收同一视频所有帧的YOLO检测结果，
通过三维度时序分析做出视频级事件判定。
"""

# 判定策略参数
VIDEO_DECISION_CFG = {
    'min_det_frame_ratio': 0.10,   # 检出帧占比最低门槛 (10%)
    'min_det_frames': 3,           # 最少检出帧数 (绝对值)
    'confidence_weight': True,     # 是否用平均置信度加权
    'temporal_window': 5,          # 时序一致性滑动窗口大小
}

def compute_temporal_consistency(frame_hits, total_frames, window=5):
    """
    计算时序一致性分数
    在滑动窗口内，连续检出的帧占比
    """
    if total_frames < window:
        return 1.0 if len(frame_hits) > 0 else 0.0

    hit_set = set(frame_hits)
    consistent_windows = 0
    total_windows = total_frames - window + 1

    for t in range(total_windows):
        hits_in_window = sum(1 for i in range(t, t + window) if i in hit_set)
        if hits_in_window >= window // 2:  # 窗口内至少半数帧检出
            consistent_windows += 1

    return consistent_windows / total_windows

def video_level_decision(per_frame_detections, total_frames, cfg):
    """
    视频级聚合判定

    Args:
        per_frame_detections: {cls_id: [(frame_idx, conf), ...]}
        total_frames: 视频总帧数
        cfg: 判定配置

    Returns:
        主事件类别, 综合评分, 所有候选事件列表
    """
    events = []
    for cls_id, hits in per_frame_detections.items():
        frame_indices = [h[0] for h in hits]
        confidences = [h[1] for h in hits]

        n_det = len(set(frame_indices))
        r_c = n_det / max(total_frames, 1)                    # 检出帧比例
        tau_c = compute_temporal_consistency(                   # 时序一致性
            frame_indices, total_frames, cfg['temporal_window'])
```

```
        conf_c = sum(confidences) / len(confidences)          # 平均置信度

        score = r_c * tau_c * conf_c  # 三维度融合评分

        if n_det >= cfg['min_det_frames'] and r_c >= cfg['min_det_frame_ratio']:
            events.append({
                'cls_id': cls_id,
                'score': score,
                'frame_ratio': r_c,
                'temporal_consistency': tau_c,
                'avg_confidence': conf_c,
                'det_frames': n_det,
            })

    events.sort(key=lambda x: x['score'], reverse=True)
    return events[0] if events else None, events
```

4.6.6 工作位置

TVAD模块**不嵌入模型结构**，而是工作在推理阶段的**后处理层**。这是刻意设计：

1. **训练阶段**：YOLO模型正常训练，学习逐帧检测能力（SADR+BDFR提升帧级精度）
2. **推理阶段**：YOLO逐帧检测后，TVAD对同一视频的结果做时序聚合判定

```
训练 → 不涉及TVAD（帧级检测训练）
推理 → YOLO逐帧检测 → TVAD聚合 → 视频级事件判定
```

这一设计的好处：

- 训练不受视频长度约束（帧级loss正常计算）
- TVAD无需额外训练数据或标签
- 可以独立调整TVAD参数而不影响模型权重
- 与现有YOLO训练流程完全兼容

4.6.7 正向作用分析

效果	分析
降低误报率	偶发误检帧占比低、时序一致性低 → 自动被过滤
提升逆行检出	通过时序分析捕获运动方向信息，弥补单帧检测的不足
确认违停事件	多帧持续检出 + 高时序一致性 → 确认非临时停靠
可解释性	输出"检出帧比例/时序一致性/平均置信度"三维评分，结果可追溯
零额外开销	仅统计计算，不增加模型参数或推理延迟
实际可用	输出直接回答"这段视频是什么事件"，贴合实际监控需求

五、训练方案

5.1 训练环境

项目	配置
GPU	NVIDIA RTX 3090 / 4090 (24GB显存)
CUDA	11.8+
Python	3.10+
PyTorch	2.0+
Ultralytics	8.3+ (支持YOLO11)

5.2 训练超参数

```python
# 文件: scripts/train.py
from ultralytics import YOLO

def train():
    # 加载改进模型配置
    model = YOLO("models/yolov11-road-anomaly.yaml")

    # 加载预训练权重
    model = model.load("yolo11m.pt")

    # 训练配置
    results = model.train(
        data="dataset/road_anomaly.yaml",

        # 基本参数
        epochs=200,
        imgsz=640,
        batch=16,

        # 优化器
        optimizer="AdamW",
        lr0=0.001,
        lrf=0.01,          # 最终学习率 = lr0 * lrf
        momentum=0.937,
        weight_decay=0.0005,
        warmup_epochs=3,
        warmup_momentum=0.8,

        # 数据增强
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
```

```
        degrees=0.0,
        translate=0.1,
        scale=0.5,
        shear=0.0,
        perspective=0.001,
        flipud=0.0,
        fliplr=0.5,
        mosaic=1.0,
        mixup=0.15,
        copy_paste=0.1,

        # 训练策略
        patience=30,        # 早停耐心值
        save_period=10,     # 每10轮保存
        device=0,           # GPU 0
        workers=8,
        amp=True,           # 混合精度训练

        # 保存
        project="runs/road_anomaly",
        name="yolov11m_improved",
        exist_ok=True,
    )

if __name__ == "__main__":
    train()
```

5.3 学习率调度

```
学习率变化曲线:

lr  ↑
    │     warmup         cosine annealing
0.001├──────/──────────────────────────────
    │    /          \
    │   /             \
    │  /                \
    │ /                   \
0.00001├/                     \────────────
    │
    └──────┬──────────────────────────────── epochs
           3                              200
```

六、实验方案

6.1 评价指标

6.1.1 帧级检测指标

指标	公式	说明
Precision (P)	$P = \frac{TP}{TP+FP}$	查准率
Recall (R)	$R = \frac{TP}{TP+FN}$	查全率
mAP@0.5	各类AP@IoU=0.5的均值	主要指标
mAP@0.5:0.95	各类AP@IoU=0.5~0.95的均值	综合指标
FPS	每秒处理帧数	实时性指标
参数量 (Params)	模型总参数数	模型复杂度
计算量 (GFLOPs)	浮点运算量	计算开销

6.1.2 视频级判定指标

指标	公式	说明
视频分类准确率 (V-Acc)	$\frac{\text{正确判定的视频数}}{\text{总视频数}}$	视频级事件判定的准确度
视频级精确率 (V-Precision)	各类别视频判定的精确率均值	减少视频级误报
视频级召回率 (V-Recall)	各类别视频判定的召回率均值	减少视频级漏报
视频级F1 (V-F1)	$\frac{2 \cdot \text{V-P} \cdot \text{V-R}}{\text{V-P}+\text{V-R}}$	综合评价视频级判定能力
误报抑制率	$1 - \frac{\text{聚合后误报数}}{\text{帧级误报数}}$	TVAD对误检的过滤能力

> **核心评价思路**：帧级指标评估YOLO检测器的质量（SADR+BDFR），视频级指标评估系统的实际可用性（TVAD）。

6.2 消融实验设计

> 验证两个创新点各自以及组合的有效性

实验编号	模型配置	SADR	BDFR	TVAD	预期结果
A0	YOLOv11m (Baseline)	✗	✗	✗	基线结果 (帧级+视频级)
A1	YOLOv11m + SADR	✓	✗	✗	多尺度目标检测提升，尤其小目标(抛洒物)
A2	YOLOv11m + BDFR	✗	✓	✗	复杂背景误检降低，精确率提升

实验编号	模型配置	SADR	BDFR	TVAD	预期结果
A3	YOLOv11m + SADR + BDFR	✓	✓	✗	帧级最佳综合性能
A4	YOLOv11m + SADR + BDFR + TVAD	✓	✓	✓	视频级判定最优，误报率大幅下降

消融实验结果表格（预期）：

实验	mAP@0.5	mAP@0.5:0.95	V-Acc	V-F1	Params(M)	GFLOPs	FPS
A0 (Baseline)	~75.0%	~50.0%	~68%	~0.65	~20.1	~68.0	~85
A1 (+SADR)	~78.8%	~53.5%	~72%	~0.70	~20.8	~70.2	~79
A2 (+BDFR)	~78.2%	~53.0%	~73%	~0.71	~20.3	~68.6	~82
A3 (+SADR+BDFR)	~81.0%	~56.2%	~76%	~0.74	~21.0	~70.8	~76
A4 (+TVAD)	~81.0%	~56.2%	**~92%**	**~0.90**	~21.0	~70.8	~76

> **关键发现 (预期)**：A3→A4 帧级指标不变（TVAD不影响YOLO模型），但视频级准确率(V-Acc)从76%跃升至92%——这验证了时序聚合判定对最终事件识别的巨大提升。

```python
# 文件: scripts/ablation_study.py
"""消融实验脚本"""
from ultralytics import YOLO
import json

configs = {
    "A0_baseline": "models/yolov11m.yaml",
    "A1_sadr": "models/yolov11m-sadr.yaml",
    "A2_bdfr": "models/yolov11m-bdfr.yaml",
    "A3_both": "models/yolov11-road-anomaly.yaml",
}

results_all = {}

for name, cfg in configs.items():
    print(f"\n{'='*60}")
    print(f"Running ablation experiment: {name}")
    print(f"{'='*60}")

    model = YOLO(cfg)
    model = model.load("yolo11m.pt")

    results = model.train(
        data="dataset/road_anomaly.yaml",
        epochs=200,
        imgsz=640,
```

```
        batch=16,
        device=0,
        project="runs/ablation",
        name=name,
        exist_ok=True,
    )

    # 评估
    metrics = model.val(data="dataset/road_anomaly.yaml")

    results_all[name] = {
        "mAP50": float(metrics.box.map50),
        "mAP50_95": float(metrics.box.map),
        "precision": float(metrics.box.mp),
        "recall": float(metrics.box.mr),
    }

# 保存结果
with open("runs/ablation/ablation_results.json", "w") as f:
    json.dump(results_all, f, indent=2)

print("\n消融实验结果汇总:")
for name, res in results_all.items():
    print(f"  {name}: mAP50={res['mAP50']:.4f}, mAP50-95={res['mAP50_95']:.4f}")
```

6.3 对比实验设计

> 与主流检测模型进行横向对比

模型	来源	类型
YOLOv5m	Ultralytics	经典YOLO
YOLOv8m	Ultralytics	前代YOLO
YOLOv11m	Ultralytics	当代基线
RT-DETR-l	Baidu	Transformer检测器
Ours (改进YOLOv11m)	本文	提出方法

对比实验结果表格（预期）：

模型	mAP@0.5	mAP@0.5:0.95	V-Acc	V-F1	Params(M)	GFLOPs	FPS
YOLOv5m	~72.0%	~47.5%	~60%	~0.58	21.2	49.0	~95
YOLOv8m	~76.2%	~51.3%	~67%	~0.64	25.9	78.9	~80
YOLOv11m	~75.0%	~50.0%	~65%	~0.62	20.1	68.0	~85
RT-DETR-l	~74.5%	~52.0%	~66%	~0.63	32.0	110.0	~45

模型	mAP@0.5	mAP@0.5:0.95	V-Acc	V-F1	Params(M)	GFLOPs	FPS
Ours	**~81.0%**	**~56.2%**	**~92%**	**~0.90**	21.0	70.8	**~76**

> **注**：对比模型的V-Acc/V-F1使用同一TVAD参数在各自帧级检测结果上聚合，体现帧级精度对视频级判定的影响。本文方法(Ours)由于帧级精度更高(SADR+BDFR)，视频级聚合(TVAD)的效果也显著更优。

```python
# 文件: scripts/comparison_experiment.py
"""对比实验脚本"""
from ultralytics import YOLO

models = {
    "yolov5m": "yolov5m.pt",
    "yolov8m": "yolov8m.pt",
    "yolov11m": "yolo11m.pt",
    "rt-detr-l": "rtdetr-l.pt",
    "ours": "models/yolov11-road-anomaly.yaml",
}

for name, weight in models.items():
    print(f"\n训练模型: {name}")

    if name == "ours":
        model = YOLO(weight)
        model = model.load("yolo11m.pt")
    else:
        model = YOLO(weight)

    model.train(
        data="dataset/road_anomaly.yaml",
        epochs=200,
        imgsz=640,
        batch=16,
        device=0,
        project="runs/comparison",
        name=name,
        exist_ok=True,
    )

    metrics = model.val(data="dataset/road_anomaly.yaml")
    print(f"  {name}: mAP50={metrics.box.map50:.4f}")
```

6.4 各类别检测精度分析

类别	特点	检测难度	改进效果预期
抛洒物 (debris)	小目标、形态多样、静态	★★★★★	SADR自动路由至小感受野分支，保留细粒度特征；TVAD过滤误检

类别	特点	检测难度	改进效果预期
违停 (illegal_parking)	中大目标、静止持续	★★☆☆☆	BDFR抑制路面/车道线背景干扰；TVAD通过持续性确认非临时停靠
逆行 (retrograde)	中目标、运动状态	★★★☆☆	SADR+BDFR协同提升帧级精度；TVAD时序分析捕获运动方向信息

6.5 视频级判定专项实验

> 验证TVAD模块的各维度贡献

实验	聚合策略	V-Acc	V-F1	说明
V0	无聚合 (取最大帧)	~60%	~0.57	最简单baseline，受误检影响大
V1	多数投票	~78%	~0.75	统计检出最多的类别
V2	平均置信度	~82%	~0.80	按平均conf排序
V3	帧比例 × 置信度	~87%	~0.85	两维度融合
V4	**帧比例 × 时序一致性 × 置信度**	**~92%**	**~0.90**	**TVAD完整方案**

七、逆行检测特殊处理

7.1 方案说明

逆行检测与普通目标检测不同，需要判断**车辆运动方向**。本方案采用"帧级检测 + 视频级判定"协同策略：

```
阶段1：帧级目标检测 (改进YOLOv11)
  └ 逐帧检测车辆并判断是否为逆行
  └ 模型学习逆行车辆的视觉特征(面向反方向、异常车道位置等)

阶段2：视频级时序判定 (TVAD)
  └ 聚合同一视频中逆行类别的检出帧
  └ 分析时序一致性：连续帧中是否持续检出逆行
  └ 高时序一致性 → 确认逆行事件
  └ 零星检出 → 可能误检，过滤
```

为什么这种策略更可靠？

传统方案	本文方案	优势
单帧检测+光流运动估计	帧级检测+TVAD视频聚合	无需额外光流计算，利用检测结果的时序统计隐式捕获运动信息
需要目标跟踪模块	不需要跟踪	更轻量，TVAD统计即可
单帧容易将正常车辆误判为逆行	视频级聚合过滤偶发误检	鲁棒性更强

7.2 数据标注策略

由于训练数据中的逆行视频已经包含了逆行场景，在标注时直接将逆行车辆标注为 `retrograde` 类别。模型会学习到逆行车辆在画面中的**典型特征**（如面向与车流方向相反、出现在异常车道位置等），从而实现端到端检测。

7.3 视频级逆行确认的时序逻辑

```
逆行视频典型时序模式:

  帧1   帧2   帧3   帧4   帧5   帧6   帧7   帧8   帧9   帧10
  ✗     ✗     ✓     ✓     ✓     ✓     ✓     ✓     ✗     ✗
                    ↑逆行车辆进入画面          ↑驶出画面

  → 检出帧比例 r = 6/10 = 60%     ✓ (≥8%)
  → 时序一致性 τ ≈ 0.85            ✓ (连续检出)
  → 平均置信度 conf ≈ 0.78        ✓
  → 综合评分 S = 0.60 × 0.85 × 0.78 = 0.398
  → 判定: 逆行事件 ✅

误检情况:
  帧1   帧2   帧3   帧4   帧5   帧6   帧7   帧8   帧9   帧10
  ✗     ✓     ✗     ✗     ✗     ✓     ✗     ✗     ✗     ✗
        ↑偶发误检              ↑偶发误检

  → 检出帧比例 r = 2/10 = 20%     ✓ (≥8%)
  → 时序一致性 τ ≈ 0.15            ✗ (不连续)
  → 综合评分 S = 0.20 × 0.15 × 0.6 = 0.018
  → 判定: 得分过低，过滤 ❌ (正确过滤了误检)
```

八、系统部署方案

8.1 PyQt5 图形界面

```python
# 文件: app/main_ui.py (核心框架)
"""
道路异常事件检测系统 - PyQt5界面
功能:
  1. 选择视频文件进行检测
  2. 摄像头实时检测
  3. 检测结果显示与日志记录
  4. 输出三类异常事件: 抛洒物/违停/逆行
"""
import sys
import cv2
import time
import numpy as np
from pathlib import Path
from PyQt5.QtCore import *
from PyQt5.QtWidgets import *
from PyQt5.QtGui import *
from ultralytics import YOLO


class DetectionThread(QThread):
    """检测线程"""
    frame_signal = pyqtSignal(np.ndarray)
    log_signal = pyqtSignal(str)

    def __init__(self, source, model_path):
        super().__init__()
        self.source = source
        self.model_path = model_path
        self.running = True

    def run(self):
        model = YOLO(self.model_path)
        cap = cv2.VideoCapture(self.source)

        class_names = {0: "抛洒物", 1: "机动车违停", 2: "逆行"}
        colors = {0: (0, 0, 255), 1: (0, 165, 255), 2: (255, 0, 0)}

        while self.running and cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            # 模型推理
            results = model(frame, conf=0.5, iou=0.45)

            # 绘制检测结果
            for r in results:
                for box in r.boxes:
                    cls_id = int(box.cls[0])
                    conf = float(box.conf[0])
                    x1, y1, x2, y2 = map(int, box.xyxy[0])

                    color = colors.get(cls_id, (0, 255, 0))
                    label = f"{class_names.get(cls_id, '未知')} {conf:.2f}"

                    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
                    cv2.putText(frame, label, (x1, y1 - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
```

```
                    # 发送日志
                    timestamp = time.strftime('%Y-%m-%d %H:%M:%S')
                    self.log_signal.emit(
                        f"[{timestamp}] 检测到: {class_names.get(cls_id)} "
                        f"(置信度: {conf:.2f})"
                    )

            self.frame_signal.emit(frame)

        cap.release()

    def stop(self):
        self.running = False


class MainWindow(QMainWindow):
    """主窗口"""
    def __init__(self):
        super().__init__()
        self.setWindowTitle("道路异常事件检测系统")
        self.setGeometry(100, 100, 1400, 900)
        self.model_path = "runs/road_anomaly/yolov11m_improved/weights/best.pt"
        self.det_thread = None
        self.init_ui()

    def init_ui(self):
        central = QWidget()
        self.setCentralWidget(central)
        layout = QVBoxLayout(central)

        # 标题
        title = QLabel("🚦 道路异常事件检测系统 (抛洒物 / 违停 / 逆行)")
        title.setAlignment(Qt.AlignCenter)
        title.setStyleSheet("font-size:28px; font-weight:bold; padding:10px;")
        layout.addWidget(title)

        # 视频显示区
        self.video_label = QLabel("请选择视频或开启摄像头")
        self.video_label.setAlignment(Qt.AlignCenter)
        self.video_label.setMinimumSize(960, 540)
        self.video_label.setStyleSheet("border:2px solid #ccc; background:#f0f0f0;")
        layout.addWidget(self.video_label)

        # 按钮栏
        btn_layout = QHBoxLayout()

        self.btn_file = QPushButton("📁 选择视频文件")
        self.btn_file.clicked.connect(self.open_file)
        btn_layout.addWidget(self.btn_file)

        self.btn_camera = QPushButton("📷 实时检测")
        self.btn_camera.clicked.connect(self.start_camera)
        btn_layout.addWidget(self.btn_camera)

        self.btn_stop = QPushButton("⏹ 停止检测")
        self.btn_stop.clicked.connect(self.stop_detection)
        btn_layout.addWidget(self.btn_stop)

        self.btn_quit = QPushButton("❌ 退出")
        self.btn_quit.clicked.connect(self.close)
        btn_layout.addWidget(self.btn_quit)
```

```
        layout.addLayout(btn_layout)

        # 日志区
        self.log_browser = QTextBrowser()
        self.log_browser.setMaximumHeight(200)
        layout.addWidget(self.log_browser)

    def open_file(self):
        path, _ = QFileDialog.getOpenFileName(
            self, "选择视频", "", "视频文件 (*.mp4 *.avi *.mkv)")
        if path:
            self.start_detection(path)

    def start_camera(self):
        self.start_detection(0)

    def start_detection(self, source):
        self.stop_detection()
        self.det_thread = DetectionThread(source, self.model_path)
        self.det_thread.frame_signal.connect(self.update_frame)
        self.det_thread.log_signal.connect(self.append_log)
        self.det_thread.start()

    def stop_detection(self):
        if self.det_thread and self.det_thread.isRunning():
            self.det_thread.stop()
            self.det_thread.wait()

    def update_frame(self, frame):
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        h, w, ch = rgb.shape
        img = QImage(rgb.data, w, h, ch * w, QImage.Format_RGB888)
        pixmap = QPixmap.fromImage(img).scaled(
            self.video_label.size(), Qt.KeepAspectRatio)
        self.video_label.setPixmap(pixmap)

    def append_log(self, text):
        self.log_browser.append(text)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())
```

8.2 推理与导出

```
# 文件: scripts/inference.py
"""推理脚本 —— 支持帧级推理和视频级推理两种模式"""
from ultralytics import YOLO

# 加载最佳模型
model = YOLO("runs/road_anomaly/yolov11m_improved/weights/best.pt")
```

```python
# ====== 模式1: 帧级推理 (传统) ======
results = model.predict(source="test_images/", conf=0.5, save=True)

# ====== 模式2: 视频级推理 (TVAD聚合判定) ======
# 逐帧检测后聚合，输出视频级事件类型
# python scripts/inference.py --source test.mp4 --video
# 输出示例:
#   test.mp4 → 判定结果: 逆行 (置信度: 0.87, 检出帧占比: 45.2%)
#   结果保存: runs/inference/video_report.json

# ====== 模式3: 批量视频推理 ======
# python scripts/inference.py --source videos_dir/ --video
# 输出:
#   video_01.mp4 → 抛洒物   (conf=0.872, ratio=35.2%)
#   video_02.mp4 → 逆行     (conf=0.791, ratio=28.6%)
#   video_03.mp4 → 机动车违停 (conf=0.834, ratio=62.1%)

# 模型导出 (部署用)
model.export(format="onnx", imgsz=640, simplify=True)
model.export(format="engine", imgsz=640, half=True)  # TensorRT
```

九、项目文件结构

```
抛洒物检测/
├── data/                          # 原始数据
│   ├── 抛洒物/                     # ~180个视频
│   ├── 机动车违停/                 # ~100个视频
│   ├── 逆行/                       # ~50个视频
│   └── images/                    # 抛洒物图片 ~800张
│
├── dataset/                       # 处理后数据集
│   ├── road_anomaly.yaml          # 数据集配置
│   ├── images/
│   │   ├── train/                 # 训练集图片
│   │   ├── val/                   # 验证集图片
│   │   └── test/                  # 测试集图片
│   └── labels/
│       ├── train/                 # 训练集标签
│       ├── val/                   # 验证集标签
│       └── test/                  # 测试集标签
│
├── models/                        # 模型配置
│   ├── yolov11-road-anomaly.yaml  # 完整改进模型
│   ├── yolov11m.yaml              # 基线模型
│   ├── yolov11m-sadr.yaml         # 仅SADR
│   ├── yolov11m-bdfr.yaml         # 仅BDFR
│   └── modules/
│       ├── sadr.py                # 创新点1: SADR模块
│       └── bdfr.py                # 创新点2: BDFR模块
│
├── scripts/                       # 脚本
│   ├── extract_frames.py          # 视频抽帧
│   ├── split_dataset.py           # 数据集划分
│   ├── train.py                   # 训练脚本
```

```
│   ├── inference.py              # 推理脚本
│   ├── ablation_study.py         # 消融实验
│   ├── comparison_experiment.py  # 对比实验
│   └── evaluate.py               # 评估脚本
│
├── app/                           # 部署应用
│   ├── main_ui.py                # PyQt5界面主程序
│   └── template/                 # UI资源文件
│
├── runs/                          # 实验结果
│   ├── road_anomaly/             # 主实验
│   ├── ablation/                 # 消融实验
│   └── comparison/               # 对比实验
│
└── docs/                          # 文档
    └── 实现方案.md
```

十、实施计划与时间线

阶段	任务	时间	产出
Phase 1	数据准备	1-2周	标注好的三类数据集(~6800帧)
Phase 2	基线训练	1周	YOLOv11m 基线结果
Phase 3	创新点实现	2周	SADR + BDFR 模块代码
Phase 4	改进模型训练	1-2周	改进模型最佳权重
Phase 5	消融实验	1周	4组消融实验结果
Phase 6	对比实验	1周	5组对比实验结果
Phase 7	系统部署	1周	PyQt5检测系统
Phase 8	论文撰写	2-3周	完整论文

总计预估: 10-13周

十一、论文章节对应关系

论文章节	对应实现内容
第1章 绪论	研究背景、意义、技术挑战
第2章 相关工作	YOLOv11综述、动态路由/核选择综述、背景建模/注意力机制综述

论文章节	对应实现内容
第3章 方法	两个创新模块（SADR+BDFR）设计与实现
第4章 实验	数据集介绍、消融实验、对比实验、可视化分析
第5章 系统实现	PyQt5检测系统、部署方案
第6章 总结与展望	贡献总结、未来工作

十二、关键创新点总结

创新点1：SADR（尺度自适应动态路由模块）

项目	内容
解决问题	三类目标尺度差异极大（抛洒物8px vs 违停车256px），固定感受野无法兼顾
核心思想	逐像素预测尺度亲和力权重，动态路由到不同膨胀率的卷积分支
关键创新	① 像素级路由（非通道级）；② 条带池化捕获道路方向性先验；③ 统一膨胀卷积等参数
与已有方法区别	SKNet是全图一个通道权重；ASPP是固定并行无路由；DCN是偏移无尺度建模
嵌入位置	Backbone末端（SPPF之后）
预期提升	mAP@0.5 提升约 3-4%，小目标AP提升最显著，新增参数 < 3%

创新点2：BDFR（背景解耦特征精炼模块）

项目	内容
解决问题	道路背景高度结构化（路面/车道线/护栏），传统注意力无法区分前景异常与背景
核心思想	学习背景原型向量，基于特征与原型的偏离度生成注意力——偏离大则增强，偏离小则抑制
关键创新	① 可学习背景原型（端到端训练）；② 偏离度驱动注意力（非统计量驱动）；③ EMA稳定更新

项目	内容
与已有方法区别	SE/CBAM只用统计量不理解"什么是背景"；异常检测方法需独立模型不可集成
嵌入位置	Neck层各特征融合节点之后
预期提升	mAP@0.5 提升约 3%，精确率（Precision）提升显著，误检率大幅下降

创新点3：TVAD（时序感知视频聚合判定模块）

项目	内容
解决问题	单帧检测无法可靠判定视频级事件类型，偶发误检导致误报，逆行/违停需时序确认
核心思想	对同一视频的逐帧检测结果进行时序聚合，融合检出帧比例、时序一致性、平均置信度三个维度评分
关键创新	① 三维度融合评分(帧比例×时序一致性×置信度)；② 滑动窗口时序一致性分析；③ 自适应类别判定阈值
与已有方法区别	多数投票无时序建模；LSTM需额外训练且不可解释；平均置信度忽略持续性
工作位置	推理阶段后处理（不改模型结构，不增加训练负担）
预期提升	视频级准确率(V-Acc)从76%→92%，误报抑制率>80%，无额外推理开销

三者协同关系

```
┌──────────────────────────────────────────────────────────────────────┐
│                          三个创新点的协同关系                          │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │ 空间维度（帧级）                                              │   │
│   │                                                              │   │
│   │   SADR ──→ 解决"在哪个尺度看"                                 │   │
│   │   │        每个空间位置自动选择匹配目标大小的感受野            │   │
│   │   │                                                          │   │
│   │   ↕ 互补                                                     │   │
│   │   │                                                          │   │
│   │   BDFR ──→ 解决"看到的是什么"                                 │   │
│   │            显式区分背景与异常前景                              │   │
│   │                                                              │   │
│   │   → 两者共同提升帧级检测精度（mAP提升 ~6%）                   │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                  │                                   │
│                                  │                                   │
│                                  ▼                                   │
```

```
│  ┌──────────────────────────────────────────────────────────────┐
│  │ 时间维度 (视频级)                                              │
│  │                                                               │
│  │   TVAD ──→ 解决"如何判定事件类型"                              │
│  │            聚合多帧结果，做出视频级事件判定                      │
│  │                                                               │
│  │   → 帧级精度越高(SADR+BDFR)，TVAD聚合效果越好                  │
│  │   → V-Acc从76%跃升至92%                                       │
│  └──────────────────────────────────────────────────────────────┘
│
│ 总结:
│   SADR+BDFR → 帧级精度基础 (空间感知)
│   TVAD → 视频级判定输出 (时序聚合)
│   空间精度 × 时序聚合 = 可靠的视频级事件检测
└──────────────────────────────────────────────────────────────────
```

> **SADR** 解决"在哪个尺度看"——每个空间位置自动选择匹配目标大小的感受野；**BDFR** 解决"看到的是什么"——显式区分背景与异常前景；**TVAD** 解决"这段视频是什么事件"——聚合时序信息做出视频级判定。三者层层递进：SADR+BDFR提供高质量的帧级检测基础，TVAD在此基础上通过时序聚合实现可靠的视频级事件分类。预期帧级 mAP@0.5 提升约 **6%**，视频级准确率(V-Acc)提升约 **24%**。

十三、风险与应对

风险	概率	应对策略
数据量不足	中	增加数据增强强度、使用半监督学习
逆行检测难度高	高	TVAD时序聚合+提高采样率(8fps)补偿帧级不足
创新点提升不明显	中	调整SADR膨胀率/BDFR原型数K/TVAD窗口大小，可视化偏离度图分析
训练时间过长	低	使用更大GPU、减小模型规模
类别不平衡	中	过采样少数类、使用Focal Loss
数据泄露	低	已按视频分组划分，同视频帧不跨split
TVAD参数敏感	低	V0~V4实验验证各维度贡献，参数可解释可调

文档版本: v3.0 (引入视频级时序推理 + 创新点3 TVAD)
创建时间: 2026年2月13日
更新时间: 2026年2月13日
适用框架: YOLOv11 (Ultralytics)